

Temas Selectos de Física Computacional III:

Introducción a la ciencia de datos.

Clase 5: 4 Marzo 2025

Dr. Leonid Serkin

1 Introducción

En esta clase, usaremos por primera vez Scikit-learn y estudiaremos los primeros pasos hacia la clasificación de usando el conjunto de datos de cáncer.

2 Scikit-learn

Scikit-learn (<https://scikit-learn.org/stable/>) es una biblioteca de Python de código abierto que incluye una amplia gama de modelos de machine learning, preprocesamiento, validación y algoritmos de visualización. Dada su simplicidad, es una muy buena opción tanto para principiantes como para científicos de datos para construir e implementar modelos de ML.

En Scikit-learn, un conjunto de datos generalmente tiene dos componentes principales:

Entradas: "Features" or "inputs" - son las variables de nuestros datos, y pueden ser representados mediante una matriz.

Respuesta: "Target" o "output" - es la variable de salida que depende de las variables de características. Generalmente, tenemos una única columna de variable de salida y se representa mediante un vector de respuesta.

3 Conjunto de datos de cáncer desde Scikit-learn

El conjunto de datos de cáncer de mama, accesible desde Scikit-learn, es utilizado para predecir si un tumor es maligno o benigno.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
df = pd.DataFrame(data=cancer.data, columns=cancer.feature_names)
df['target'] = cancer.target # 0 = Benigno, 1 = Maligno
print(df.head())
```

3.1 Variables y su correlación con el diagnóstico

En la clase pasado, analizamos la matriz de dispersión para las primeras 4 características.

```
from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt
scatter_matrix(df.iloc[:, :4], c=df['target'], cmap='bwr')
plt.show()
```

y también vimos como obtener las correlaciones lineales de Pearson:

```
import seaborn as sns
plt.figure(figsize=(10, 10))
sns.heatmap(df.iloc[:, 0:4].corr(), annot=True, square=True, cmap='coolwarm')
plt.show()
```

3.2 Matriz de confusión

La matriz de confusión es una tabla que nos permite evaluar el rendimiento de un modelo de clasificación comparando sus predicciones con los valores reales. En el caso de una clasificación binaria, la matriz tiene la siguiente estructura:

	<i>Predicho : Negativo(0)</i>	<i>Predicho : Positivo(1)</i>
<i>Real : Negativo(0)</i>	<i>TN</i>	<i>FP</i>
<i>Real : Positivo(1)</i>	<i>FN</i>	<i>TP</i>

donde:

- TN (True Negative) - Verdaderos negativos: casos benignos correctamente clasificados como benignos.
- FP (False Positive) - Falsos positivos: casos benignos incorrectamente clasificados como malignos.
- FN (False Negative) - Falsos negativos: casos malignos incorrectamente clasificados como benignos.
- TP (True Positive) - Verdaderos positivos: casos malignos correctamente clasificados como malignos.

Para visualizar la matriz de confusión, se usa:

```
import seaborn as sns
import matplotlib.pyplot as plt
cm = [[50, 10], [5, 35]]
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.show()
```

3.3 Métricas de evaluación

A partir de la matriz de confusión, podemos calcular varias métricas importantes.

La **exactitud** (accuracy) mide la proporción de predicciones correctas sobre el total de muestras:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Esta métrica es útil cuando las clases están balanceadas, pero puede ser engañosa si hay un desbalance significativo.

La **precisión** (precision) nos indica qué tan confiable es el modelo cuando predice un caso como positivo:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Si la precisión es alta, significa que hay pocos falsos positivos (FP), es decir, el modelo no está clasificando erróneamente demasiados benignos como malignos.

La **sensibilidad** (recall), también llamada **tasa de verdaderos positivos** (TPR, True Positive Rate) responde a la pregunta: de todos los casos realmente malignos, cuántos logré identificar?.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Si el recall es bajo, significa que hay muchos falsos negativos (FN), lo cual puede ser peligroso en un diagnóstico médico, ya que tumores malignos podrían quedar sin detectar.

Y finalmente, el **F1-score** es la media armónica entre la precisión y el recall:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Es útil cuando hay desbalance de clases, ya que ofrece una mejor evaluación del modelo sin favorecer ni la precisión ni el recall en exceso.

Para calcularlos usando Scikit-learn, se usa:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy = accuracy_score(df['target'], df['prediction'])
precision = precision_score(df['target'], df['prediction'])
recall = recall_score(df['target'], df['prediction'])
f1 = f1_score(df['target'], df['prediction'])
```

En un problema de clasificación de tumores, la elección del umbral afecta directamente las métricas de evaluación. Para determinar el mejor umbral, consideramos las siguientes métricas:

- Si el objetivo es detectar todos los tumores malignos, se debe maximizar el recall para minimizar los **falsos negativos**. Esto es crucial en el diagnóstico médico, donde no detectar un cáncer puede ser crítico. Sin embargo, un umbral muy bajo puede aumentar los **falsos positivos**.
- Si se busca reducir diagnósticos erróneos y evitar pruebas innecesarias, se debe maximizar la **precisión**. Sin embargo, un umbral muy alto puede llevar a muchos **falsos negativos**, lo que es peligroso en este contexto.
- El **F1-score** equilibra precisión y recall, siendo útil cuando ambas métricas son importantes. En diagnóstico médico, normalmente se prioriza el **recall** al inicio, y luego se ajusta la precisión con pruebas adicionales.

3.4 Estudio del umbral

Para evaluar qué tan bien sirven estas variables para predecir el diagnóstico, seguimos una estrategia de **valor de umbral**.

1. Seleccionar **tres** variables del conjunto con diferente correlación con el diagnóstico: una de alta correlación, una de correlación media y una de baja correlación. Para ello calcula la matriz de correlación y ordena los valores en orden descendente. Puedes usar `df.corr()[['target']]` para obtener la correlación de todas las variables con el diagnóstico (`target`).
2. Estudia las tres variables, puedes usar `df['VARIABLE'].describe()` para analizar su distribución y valores estadísticos.
3. Dibuja las variables para tumores malignos y benignos, y usa la estimación de densidad de kernel (KDE) para visualizar su distribución.
4. El siguiente paso es evaluar qué tan bien predicen si un tumor es Maligno (M) o Benigno (B). Para hacerlo de manera sencilla, podemos comparar esta variable con el diagnóstico real (`target`) y calcular la matriz de confusión y las métricas.
5. Escoge "a ojo" un valor de umbral para cada variable que te ayudará a clasificar los tumores como malignos ($M = 1$) o benignos ($B = 0$).
6. Ahora asigna etiquetas de clasificación o predicción, esto significa que defines un `df['prediction']` tal que: si el valor de `variable` es menor o igual al umbral, predice M (Maligno). Si el valor de `variable` es mayor al umbral, predice B (Benigno).
7. Imprime las primeras 30 filas con los valores reales del diagnóstico y las predicciones generadas. Por qué hay casos con `target = 0` (Benigno) que tienen `prediction = 1` (Maligno)?
8. Define FP: casos donde el `target` real es 0 (Benigno) pero la predicción es 1 (Maligno). Define FN: casos donde el `target` real es 1 (Maligno) pero

la predicción es 0 (Benigno). Cuéntalos y encuentra cuantas predicciones fallaron?

9. Dibuja la matriz de confusión completa.
10. Evalua todas las 4 metricas que apenas vimos. Tu modelo es confiable?
11. Para probar diferentes valores de umbral, necesitamos graficar la "Precision-Recall Curve" para ver el mejor punto de equilibrio:

```
from sklearn.metrics import precision_recall_curve
true = df['target']
var = df['alguna variable']
precision, recall, thresholds = precision_recall_curve(true, var)
plt.plot(thresholds, precision[:-1], label="Precision", linestyle="--")
plt.plot(thresholds, recall[:-1], label="Recall")
plt.xlabel("Umbral")
plt.ylabel("Valor")
plt.legend()
plt.show()
```

Estudia estas gráficas para tus variables. Nota que si el recall cae abruptamente al aumentar el umbral, esto indica que esta variable no es buena para distinguir entre tumores benignos y malignos. Si la precisión tiene fluctuaciones, esto sugiere que no hay un patrón claro en cómo esta variable separa las clases.

12. **+1 punto extra:** En caso de minimizar los falsos negativos (FN), esto ayuda a evitar que pacientes con cáncer real sean mal diagnosticados como sanos. Optimiza el umbral para minimizar los FNs. Compara el desempeño entre usar una sola variable (la de mas alta correlación con target) y una combinación de dos variables, donde la segunda es supongamos la de mas baja correlación con target. Genera tus predicciones de tres formas: usando solo la primera, la segunda, y una regla combinada que clasifique como maligno solo si ambas condiciones se cumplen. Comparar los métodos para ver cuál reduce mejor los FNs sin aumentar demasiado los FPs.